Authored by: Arpit Raj, LNMIIT Jaipur
Date: October 24, 2026
© 2026 Arpit Raj. All rights reserved.

Advanced C++ OOP

Slicing, Virtual Inheritance & Ambiguity

Object Slicing

-> object slicing occurs when a derived class object is copied into a base class object by value. This causes specialized portion of child to be lost from resulting base object.

```
class Animal {
public:
    int x = 10;
};

class Dog : public Animal {
public:
    int y = 20;
};

main() {
    Dog D;
    Animal A = D;
}
```

- -> before slicing (`Dog d`): Base portion -> `x=10` , derived portion -> `y=20`
- ↓ copy by value `A=D`
- after slicing (`Animal A`): Base portion -> `x=10` , now `y` is not a part of `A` because it was dog specific

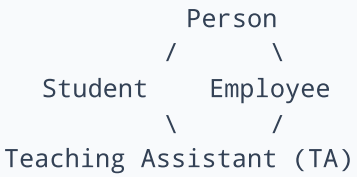
3 Cases of Object Slicing

1. `Dog d; Animal a = d;`
   - -> Slicing occurs when we copy derived object into a base object
2. `void show(Animal a) { cout << a.x; }`
   `Dog d; show(d);`
   - -> in pass by value, slicing occurs. Function receives an animal object by value so derived portion is sliced away
3. `void show(Animal &a) { cout << a.x; }`
   `Dog d; show(d);`
   - -> in pass by reference, `a` is reference to existing `d` object and not a new animal object hence original object's derived portion remains intact (NO SLICING)

> 💡 **NOTE: THE LISKOV SUBSTITUTION PRINCIPLE**
>
> The Liskov substitution principle says a derived object should be usable in place of its base object without breaking expected behaviour.
>
> **EG:**
>
> base class promises every object can fly, that base class is Bird -> now penguin inherits from it and cant fly.
> Thats why a grammatical IS-A does not guarantee a good inheritance design.

The Diamond Problem & Virtual Inheritance

Diagram: Person branches down to student and employee. Both student and employee branch down to Teaching assistant.



Here if inheritance is ordinary the `TA` contains 2 separate subobjects of person.
This duplication of subobjects is called **diamond problem**.

How is the duplication a problem

let Person have age
then `ta.age` is ambiguous as we dont know age of student or employee
so need `ta.Student::age` or `ta.Employee::age`
Thats why `ta.age` fails because 2 branches can hold different values even if logically age should have been same.

Solution: use `virtual` keyword when declaring intermediate classes inheritance

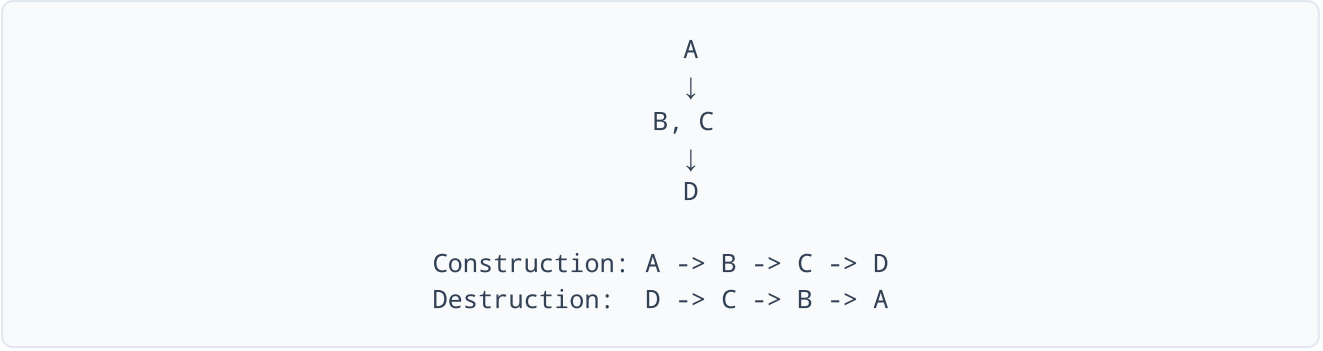
```
class Student : virtual public Person { };

class TA : public Student, public Employee { };
```

Virtual inheritance ensures that the class farthest down the line shares one common subobject of base class through both branches.

Order of Inheritance Execution

Diagram: Diamond A (top) -> B, C (middle) -> D (bottom)



1. virtual Base class
2. non virtual direct base classes in same order as in declaration
3. derived class own members
4. derived class constructor body

destruction happens as `D -> C -> B -> A`

Output Qs:

```cpp
class A {
public:
    A(int x) { cout << x << " "; }
}; // -> virtual Base class

class B : virtual public A {
public:
    B() : A(10) { cout << "B "; }
}; // -> direct non virtual base classes

class C : virtual public A {
public:
    C() : A(20) { cout << "C "; }
};

class D : public B, public C {
public:
    D() : A(30) { cout << "D "; }
}; // -> derived class
```

O/p -> 30 B C D

because `D` is the lowest class, hence its initializer `A(30)` determines how virtual `A` is constructed

📌 Note: Constructor Execution Order

Constructors of direct base classes run in order they appear in inheritance declaration.

let A, B point down to C and you write `class C : public B, public A {}`
then
B A C

(not A B C)

⚠️ WARNING: UNDEFINED BEHAVIOUR IN INITIALIZATION

Note:

```
class Test {
public:
    int a;
    int b;
    Test() : b(10), a(b) { }
};
```

even though `b(10)` written first, `a` is declared first so it wants to get filled with value first `a(b)`, but `b` isn't filled at that time, so undefined behaviour.

Name Hiding & Ambiguity

```
class A {
public:
    void show() { cout << "A show"; }
};

class B : public A {
public:
    void show() { cout << "B show"; }
};
```

now in main ->
`B obj;`
`obj.show();`
O/p -> B show

reason: both `A` and `B` have function named `show()` so name declared in `B` hides base class name through `B` / **IT DOES NOT OVERRIDE, IT HIDES. for A write obj.A::show()**
if want "A show" write: `obj.A::show()`

Interview Questions

Q1. Overload Hiding

```
class A {
public:
    void show() { cout << "A \n"; }
    void show(int x) { cout << x << "\n"; }
};

class B : public A {
public:
    void show(string s) { cout << s << "\n"; }
};
```

`B obj;`
`obj.show("hello")` // -> hello
`obj.show()` // -> Error
`obj.show(10)` // -> Error

wont operate because B hides entire base class methods named `show` during lookup through B.
Solution -> write `using A::show;` inside `class B : public A {}`. now base class overload is also exposed.

Q2. Ambiguity Question

```
class A { public: void show() { cout << "A\n"; } };
class B { public: void show() { cout << "B\n"; } };
class C : public A, public B { };
```

now
`C obj;`
`obj.show()` -> error

because C does not have a `show` to hide base overloads which are colliding.